

DATA 260 HOMEWORK 1

Grocery Supply and Recall Notices

Sharan Lourduraj | SJSU ID: 018169619 | SID4 9619

Section 0 | Personal Configuration and Domain

Value	Result
SID4	9619
PORT_BASE	8619
PREFIX	s9619
SEED	9619
VERIFY_SEED	269619
DOMAIN_ID	3
Assigned domain	Grocery supply and recall notices
Hardware	HP Pavilion; 32 GB RAM; 512 GB storage
Local model	qwen3:8b

GitHub repository: <https://github.com/Sharanjoo/data260-9619>

Implementation commit reference: 3f32f56492ce29987d9a27927ba2c96e2024f518

PART I | HTML

The entity is a Grocery Recall Notice. I created an HTML page titled **HW1-Sharan Lourduraj** for my assigned domain, **Grocery Supply and Recall Notices**. I used an `<h1>` heading because it is the largest HTML heading.

Before creating the webpage, I defined the Grocery Recall Notice fields and categories in `DOMAIN_SCHEMA.md`.

The form made as per requirements from the pdf.

Finally, I added `<script src="app.js"></script>` at the end of the HTML body to connect the JavaScript required in Part II. The completed form is shown in the screenshot pasted below.

github.com/Sharanjoo/data260-9619/blob/main/code/web_application/index.html

data260-9619 / code / web_application / index.html

Sharanjoo refactoring: organized web app and Docker files 6bf2eb · 2 days ago History

Code Blame 91 lines (80 loc) · 2.74 KB

```
1 <!doctype html>
2 <html lang="en">
3 <head>
4   <meta charset="utf-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1">
6   <meta name="description" content="Submit grocery supply and product recall notices.">
7   <title>HTML-Sharan Lourduraj</title>
8   <link rel="stylesheet" href="styles.css">
9 </head>
10 <body>
11   <div class="container">
12     <div>
13       <h1>Grocery Supply and Recall Notices</h1>
14       <p>Use this form to submit a grocery supply or product recall notice.</p>
15     </div>
16
17     <form id="noticeform" novalidate>
18       <h2>Submit a Notice</h2>
19
20       <label for="productName">Product name *</label>
21       <input
22         type="text"
23         id="productName"
```

github.com/Sharanjoo/data260-9619/blob/main/code/web_application/index.html

data260-9619 / code / web_application / index.html

Code Blame 91 lines (80 loc) · 2.74 KB

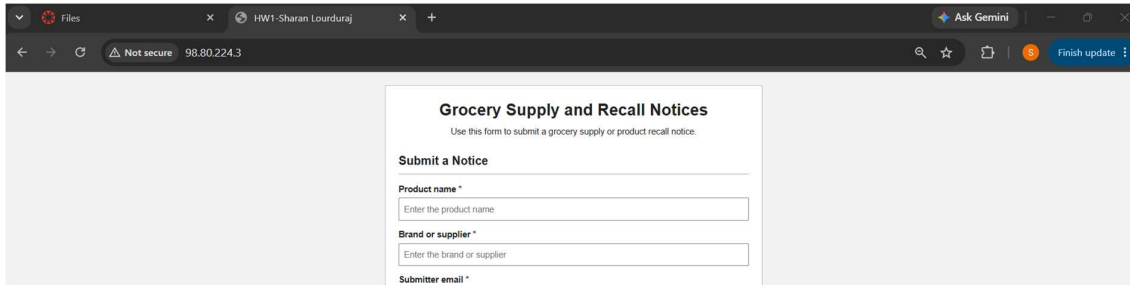
```
24       name="productName"
25       placeholder="Enter the product name"
26       autocomplete="off"
27       autofocus
28       required
29     >
30
31     <label for="brandName">Brand or supplier *</label>
32     <input
33       type="text"
34       id="brandName"
35       name="brandName"
36       placeholder="Enter the brand or supplier"
37       autocomplete="organization"
38       required
39     >
40
41     <label for="submitterEmail">Submitter email *</label>
42     <input
43       type="email"
44       id="submitterEmail"
45       name="submitterEmail"
46       placeholder="Enter your email address"
47       autocomplete="email"
48       required
49     >
50
51     <label for="category">Notice category *</label>
```

github.com/Sharanjoo/data260-9619/blob/main/code/web_application/index.html

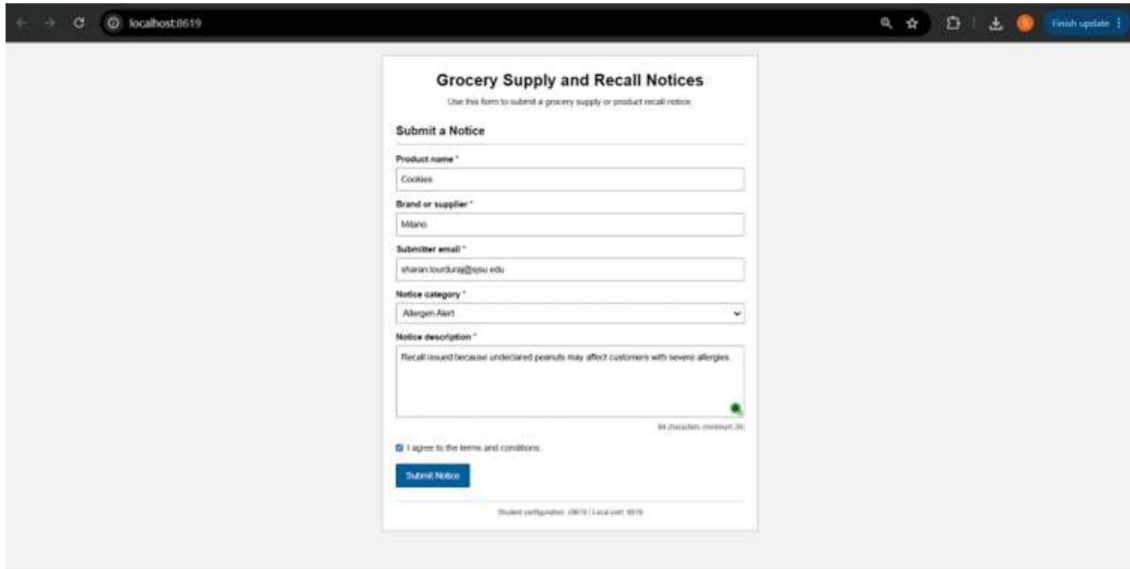
data260-9619 / code / web_application / index.html

Code Blame 91 lines (80 loc) · 2.74 KB

```
51   <label for="category">Notice category *</label>
52   <select id="category" name="category" required>
53     <option value="" selected disabled>Select a category</option>
54     <option value="food-safety-recall">Food Safety Recall</option>
55     <option value="allergen-alert">Allergen Alert</option>
56     <option value="quality-withdrawal">Quality Withdrawal</option>
57     <option value="supply-shortage">Supply Shortage</option>
58   </select>
59
60   <label for="description">Notice description *</label>
61   <textarea
62     id="description"
63     name="description"
64     rows="5"
65     minlength="26"
66     placeholder="Describe the product issue and recommended action"
67     required
68   ></textarea>
69   <p class="hint"><span id="characterCount">0</span> characters (minimum 26)</p>
70
71   <div class="terms-row">
72     <input type="checkbox" id="termsAccepted" name="termsAccepted" required>
73     <label for="termsAccepted">I agree to the terms and conditions.</label>
74   </div>
75
76   <button type="submit">Submit Notice</button>
77 </form>
78
```



To show the web page HTML page titled **HW1-Sharan Lourduraj**



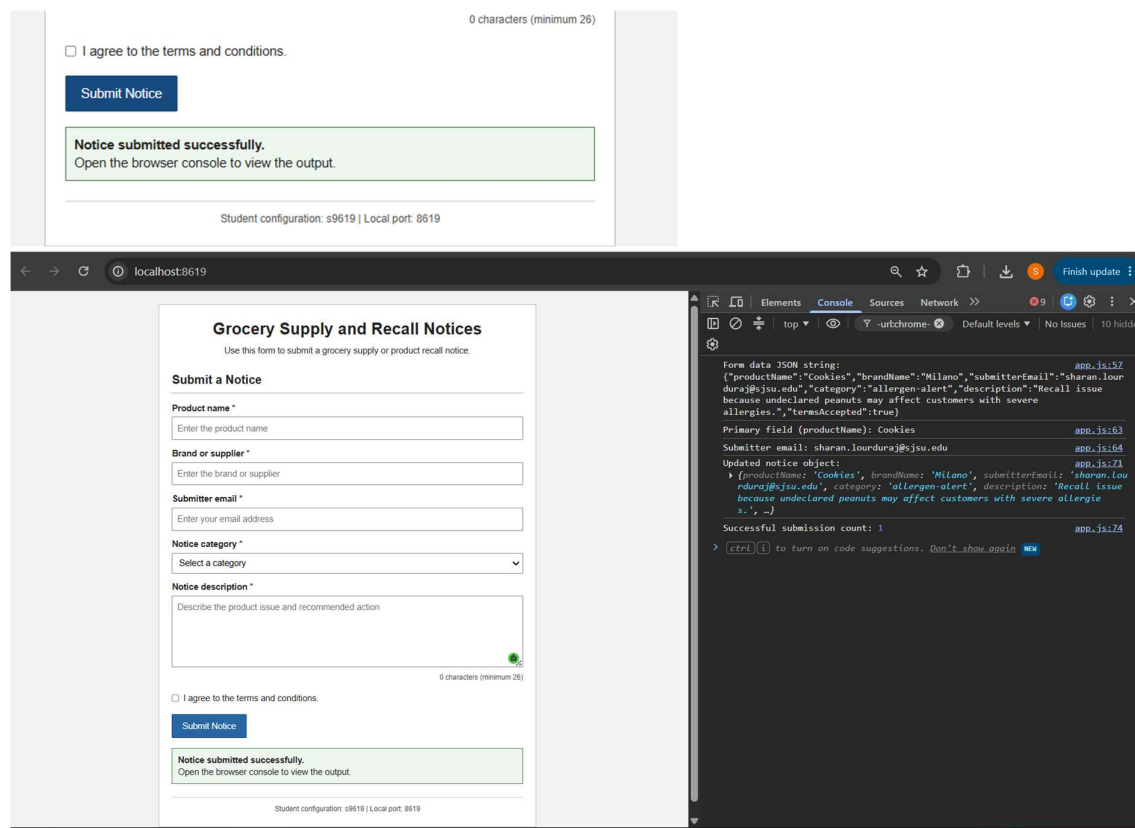
JAVASCRIPT

I created the `validateForm` arrow function to validate the form before submission. It checks that the notice description contains more than 25 characters and that the terms checkbox is selected. If either check fails, an appropriate alert is displayed.

After successful validation:

1. The form data is converted into a JSON string and logged in the console.
2. The JSON is parsed, and object destructuring extracts the product name and submitter email.
3. The spread operator creates an updated object with an ISO-formatted `submissionDate`.
4. A closure named `submissionCounter` privately tracks and logs the number of successful submissions.

The browser-console screenshot below shows the JSON string, extracted values, updated object, and submission count.



github.com/Sharanjoo/data260-9619/blob/main/code/web_application/app.js

data260-9619 / code / web_application / app.js

Sharanjoo refactoring: organized web app and Docker files 6bf2eb · 2 days ago History

Code Blame 80 lines (64 loc) · 2.32 KB

```
1 "use strict";
2
3 const form = document.getElementById("noticeform");
4 const descriptionInput = document.getElementById("description");
5 const termsInput = document.getElementById("termsAccepted");
6 const characterCount = document.getElementById("characterCount");
7 const successMessage = document.getElementById("successMessage");
8
9 // Closure: count is private and survives between successful submissions.
10 const submissionCounter = () => {
11   let count = 0;
12   return () => {
13     count += 1;
14     return count;
15   };
16 }();
17
18 // Required arrow function validation for the content and terms fields.
19 const validateForm = () => {
20   if (descriptionInput.value.trim().length <= 25) {
21     alert("The notice description must contain more than 25 characters.");
22     descriptionInput.focus();
23     return false;
24   }
25
26   if (!termsInput.checked) {
```

github.com/Sharanjoo/data260-9619/blob/main/code/web_application/app.js

data260-9619 / code / web_application / app.js

Code Blame 80 lines (64 loc) · 2.32 KB

```
25
26   if (!termsInput.checked) {
27     alert("You must agree to the terms and conditions before submitting.");
28     termsInput.focus();
29     return false;
30   }
31
32   if (!form.checkValidity()) {
33     form.reportValidity();
34     return false;
35   }
36
37   return true;
38 };
39
40 descriptionInput.addEventListener("input", () => {
41   characterCount.textContent = String(descriptionInput.value.length);
42 });
43
44 form.addEventListener("submit", (event) => {
45   event.preventDefault();
46   successMessage.hidden = true;
47
48   if (!validateForm()) {
49     return;
50   }
51
52   const formData = {
53     productName: productInput.value,
54     submitterEmail: emailInput.value,
55     notice: descriptionInput.value,
56     termsAccepted: termsInput.checked
57   };
58
59   // Convert successful form data to a JSON string and log it.
60   const jsonString = JSON.stringify(formData);
61   console.log("Form data JSON string:", jsonString);
62
63   const parsedNotice = JSON.parse(jsonString);
64
65   // Object destructuring: extract the primary field and submitter email.
66   const { productName, submitterEmail } = parsedNotice;
67   console.log("Primary field (productName):", productName);
68   console.log("Submitter email:", submitterEmail);
69
70   // Spread operator: copy the parsed object and add an ISO timestamp.
71   const datedNotice = {
72     ...parsedNotice,
73     submissionDate: new Date().toISOString(),
74   };
75   console.log("Updated notice object:", datedNotice);
76
77   const successfulSubmissions = submissionCounter();
78   console.log("Successful submission count:", successfulSubmissions);
79
80   successMessage.hidden = false;
81   form.reset();
82   characterCount.textContent = "0";
83   document.getElementById("productName").focus();
84 });
```

github.com/Sharanjoo/data260-9619/blob/main/code/web_application/app.js

data260-9619 / code / web_application / app.js

Code Blame 80 lines (64 loc) · 2.32 KB

```
53   const formData = {
54     productName: productInput.value,
55     submitterEmail: emailInput.value,
56     notice: descriptionInput.value,
57     termsAccepted: termsInput.checked
58   };
59
60   // Convert successful form data to a JSON string and log it.
61   const jsonString = JSON.stringify(formData);
62   console.log("Form data JSON string:", jsonString);
63
64   const parsedNotice = JSON.parse(jsonString);
65
66   // Object destructuring: extract the primary field and submitter email.
67   const { productName, submitterEmail } = parsedNotice;
68   console.log("Primary field (productName):", productName);
69   console.log("Submitter email:", submitterEmail);
70
71   // Spread operator: copy the parsed object and add an ISO timestamp.
72   const datedNotice = {
73     ...parsedNotice,
74     submissionDate: new Date().toISOString(),
75   };
76   console.log("Updated notice object:", datedNotice);
77
78   const successfulSubmissions = submissionCounter();
79   console.log("Successful submission count:", successfulSubmissions);
80
81   successMessage.hidden = false;
82   form.reset();
83   characterCount.textContent = "0";
84   document.getElementById("productName").focus();
85 });
```

Deployment

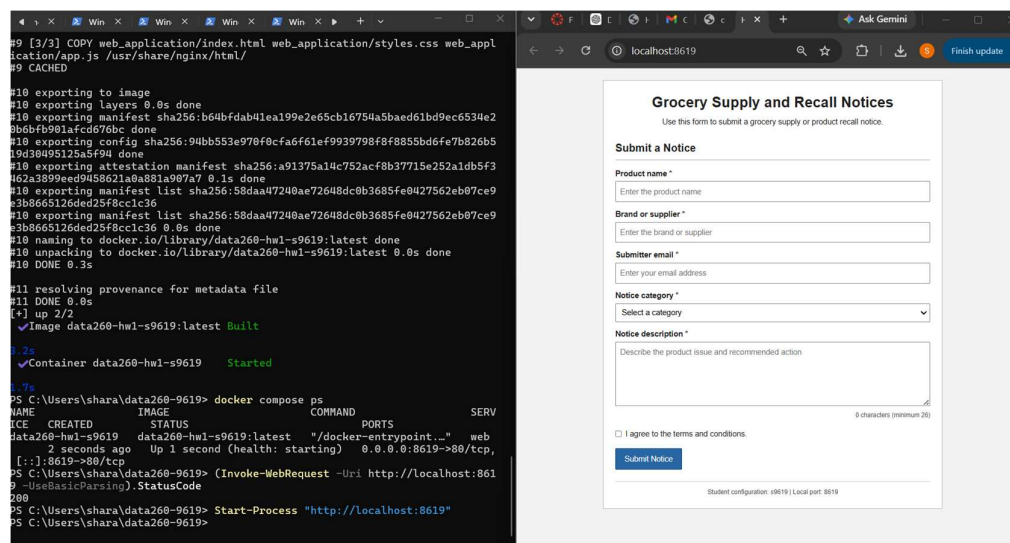
Docker | Local Deployment

I first packaged the HTML, CSS, and JavaScript files into a Docker image. The image uses Nginx as the web server. I then started a Docker container and connected my computer's port 8619 to the container's port 80.

This allowed me to open and test the application locally at:

<http://localhost:8619>

The screenshot below shows the application running successfully from the Docker container.

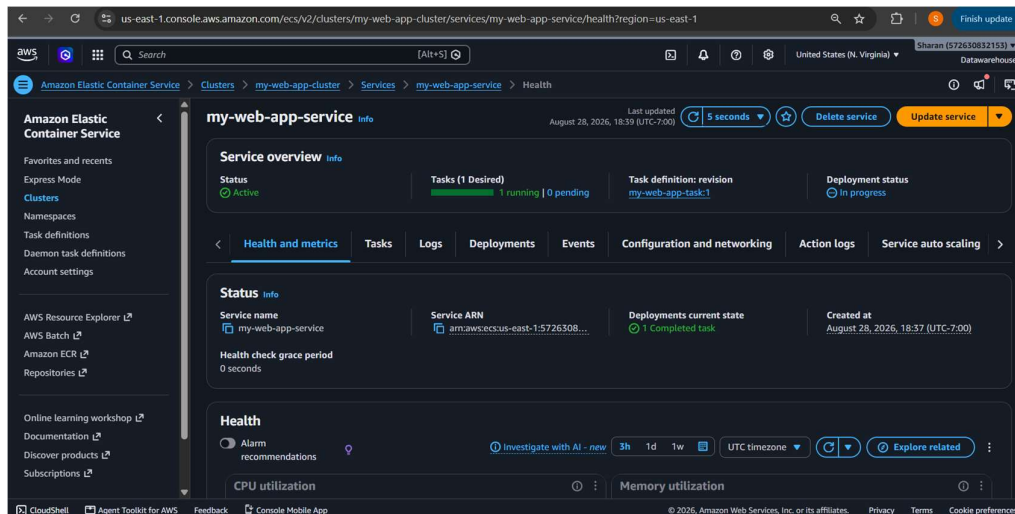


AWS ECS | Cloud Deployment

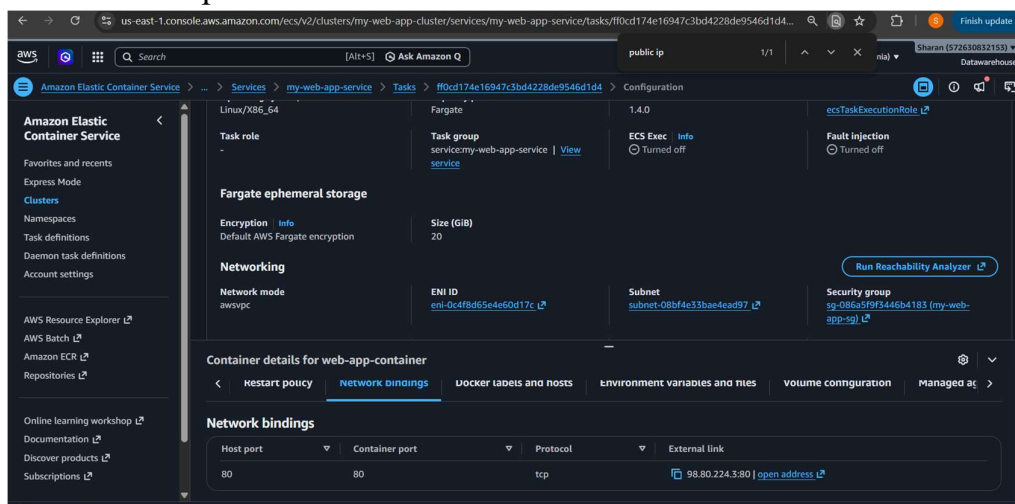
After testing locally, I uploaded the same Docker image to Amazon ECR, which stores Docker images in AWS. I then created an Amazon ECS service using AWS Fargate to run the application without managing a server.

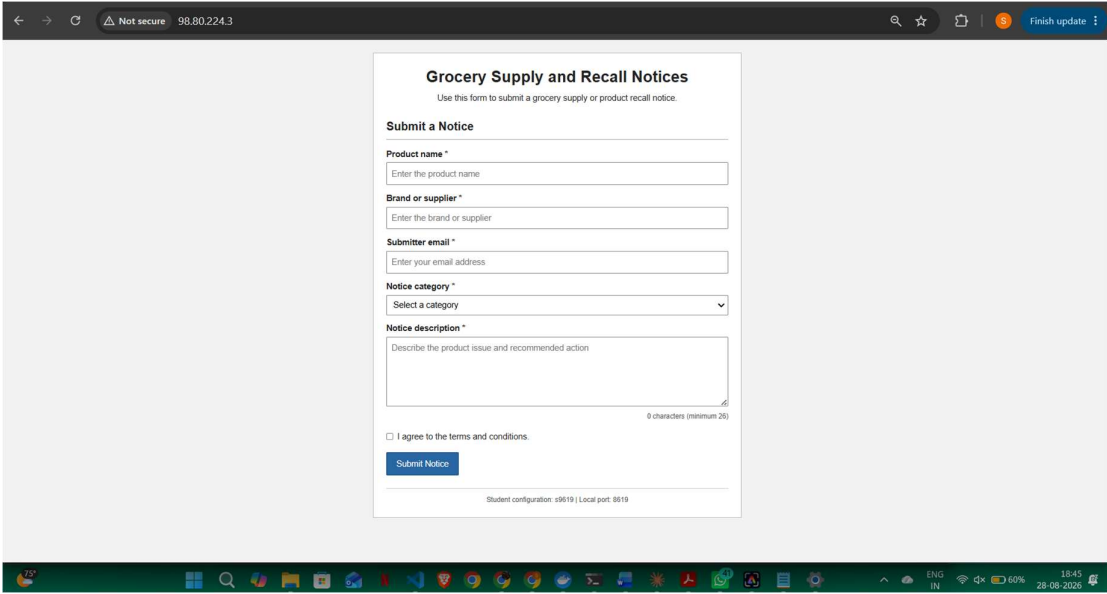
The ECS service ran exactly one task. I enabled a public IP and allowed incoming traffic on port 80, which made the application available through the internet.

The screenshots below show the ECS service with one running task and the application loaded through its public IP address. After collecting the evidence, I changed the desired task count to zero to stop the running compute resources.



Public IP: <http://98.80.224.3/>





Part 2 Agentic AI

I created a small agent workflow using the local qwen3:8b model through Ollama. The program receives a title and description, produces exactly three tags and a summary of no more than 25 words, and prints the result as valid JSON.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\shara> ollama --version
ollama version is 0.33.1
PS C:\Users\shara> ollama pull qwen3:8b
pulling manifest
pulling a3de86cd1c13: 100% 5.2 GB
verifying sha256 digest
writing manifest
success
PS C:\Users\shara> ollama run qwen3:8b "Reply with READY only."
Thinking...
Okay, the user wants me to reply with "READY" only. Let me make sure I understand the request correctly. They mentioned "Reply with READY only." So, the instruction is straightforward—they just need the word "READY" in uppercase without any additional text.

I should check if there's any hidden context or if they might be testing something. But since the query is simple, it's likely they just want a confirmation that I'm prepared or ready for something. Maybe they're starting a conversation or a task and want to confirm my status.

I need to ensure that my response strictly follows their instruction. No extra words, no explanations. Just "READY." Let me make sure there are no typos or formatting issues. The user might be using this in a script or a specific application where only "READY" is acceptable.

Also, considering the possibility of automation, perhaps they're setting up a system where "READY" is a signal. I should respond exactly as requested. No markdown, just plain text. Alright, I'll go with "READY" as the response.
...done thinking.

READY
PS C:\Users\shara>
```

Command Used

```
python scripts/recorded_run.py -- python code/agents_demo.py `
--input-file reports/hw01/cases/nondeterminism_input.json `
--model qwen3:8b `
--temperature 0.0 `
--output-json reports/hw01/raw/agent_demo_result.json
```

Workflow Explanation

1. **Planner:** The Planner reads the supplied title and description. It creates three relevant tags and a short summary based only on that input.
2. **Reviewer:** The Reviewer checks whether the tags are relevant and unique and whether the summary is accurate and within the 25-word limit. It can correct the Planner's response if necessary.
3. **Finalizer:** The Finalizer checks the final structure and guarantees that the result contains exactly three tags and one summary.
4. **Publish:** The approved result is printed as valid JSON.

The tags are generated from the input instead of being hardcoded into the agent logic.

Results

Q1. What were the three final tags?

- food recall
- allergen warning
- product safety

Q2. What was the final summary?

“Milano is recalling cookies due to undeclared peanuts, risking allergic reactions and offering refunds.”

Q3. Did the Reviewer change anything?

No. The Reviewer confirmed that the tags were distinct and supported by the input and that the summary was accurate and within the word limit.

The console screenshot below shows the Planner, Reviewer, Finalized, and Publish outputs.

```
PS C:\Users\shara\data260-9619> $r = Get-Content ".\reports\hw01\raw\agent_demo_result.json" -Raw | ConvertFrom-Json
PS C:\Users\shara\data260-9619> Write-Host " Planner Output "; [ordered]@{tags=$r.transcript.planner.tags; summary=$r.transcript.planner.summary} | ConvertTo-Json -Depth 5 -Compress
Planner Output
{"tags":["food recall","allergen warning","product safety"],"summary":"Milano is recalling cookies due to undeclared peanuts, risking allergic reactions and offering refunds."}
PS C:\Users\shara\data260-9619> Write-Host " Reviewer Output "; [ordered]@{tags=$r.transcript.reviewer.tags; summary=$r.transcript.reviewer.summary; changed=$r.transcript.reviewer.changed; explanation=$r.transcript.reviewer.explanation} | ConvertTo-Json -Depth 5 -Compress
Reviewer Output
{"tags":["food recall","allergen warning","product safety"],"summary":"Milano is recalling cookies due to undeclared peanuts, risking allergic reactions and offering refunds.", "changed":false, "explanation":"Tags and summary match the Planner\u0027s proposal. All tags are distinct, specific, and supported by the content. Summary is concise and within the word limit."}
PS C:\Users\shara\data260-9619> Write-Host " Finalized Output "; $r.final | ConvertTo-Json -Depth 5 -Compress
Finalized Output
{"tags":["food recall","allergen warning","product safety"],"summary":"Milano is recalling cookies due to undeclared peanuts, risking allergic reactions and offering refunds."}
PS C:\Users\shara\data260-9619> Write-Host " Publish Output (valid JSON) "; @{publish=$r.final} | ConvertTo-Json -Depth 5 -Compress
Publish Output (valid JSON)
{"publish":{"tags":["food recall","allergen warning","product safety"],"summary":"Milano is recalling cookies due to undeclared peanuts, risking allergic reactions and offering refunds."}}
```

Part 3 Measuring Non-Determinism

I saved one fixed input in reports/hw01/cases/nondeterminism_input.json and used it without making any changes.

Title: Milano Cookies Allergen Alert

Content: Milano is recalling packages of Cookies because undeclared peanuts may affect customers with severe allergies. Customers should not consume the product and should return it to the store for a refund.

I ran the agent pipeline 40 times:

- 20 runs at temperature 0.7
- 20 runs at temperature 0.0

Temperature controls the amount of variation in the model’s responses. Temperature 0.7 allows more variation, while temperature 0.0 produces more consistent results.

Command Used

```
python scripts/run_nondeterminism.py --model qwen3:8b --resume
```

Tag Results

Metric	Temperature 0.7	Temperature 0.0
Distinct tag sets	6	1
Tags in all 20 runs	None	allergen warning, food recall, product safety
Tags appearing in exactly one run	allergen recall	None

Latency Results

Metric	Temperature 0.7	Temperature 0.0
p50 latency	25,716.793 ms	22,887.268 ms
p95 latency	35,811.859 ms	23,351.704 ms
p99 latency	62,209.399 ms	24,772.876 ms

The p50 value represents the typical response time. The p95 and p99 values show how long the slower requests took.

Interpretation

Temperature 0.7 produced six different tag combinations, while temperature 0.0 produced only one. Therefore, two users sending the same input may receive different but related tags at temperature 0.7, while temperature 0.0 is more consistent.

Variation is acceptable for optional discovery tags, such as using product safety instead of allergen recall. However, variation is not acceptable if the model removes or changes the warning about undeclared peanuts because that information affects customer safety.

The raw results for all 40 runs were saved as JSON and CSV files in reports/hw01/raw/. The screenshot below shows the completed experiment.

```
PS C:\Users\shara\data260-9619> python scripts/run_nondeterminism.py --model qwen3:8b --resume
[2026-08-29T00:10:41Z] START nondeterminism experiment model=qwen3:8b runs_per_temperature=20 input_sha256=8a047e5f08fb54440debe3223dbc242e7f666d5205e5f1e9
1a210cf9798359f
[2026-08-29T00:11:50Z] RESULT temperature=0.7 run=1 latency_ms=68808.784 tags=["food recall", "allergen warning", "product safety"]
[2026-08-29T00:12:12Z] RESULT temperature=0.7 run=2 latency_ms=32212.497 tags=["food recall", "allergen safety", "product return"]
[2026-08-29T00:12:53Z] RESULT temperature=0.7 run=3 latency_ms=38575.28 tags=["food recall", "allergy warning", "product safety"]
[2026-08-29T00:13:27Z] RESULT temperature=0.7 run=4 latency_ms=33773.592 tags=["food recall", "allergen warning", "product safety"]
[2026-08-29T00:13:56Z] RESULT temperature=0.7 run=5 latency_ms=29609.815 tags=["food recall", "allergen safety", "product return"]
[2026-08-29T00:14:30Z] RESULT temperature=0.7 run=6 latency_ms=34075.179 tags=["food recall", "allergen warning", "product safety"]
[2026-08-29T00:14:58Z] RESULT temperature=0.7 run=7 latency_ms=27750.474 tags=["food recall", "allergen safety", "product return"]
[2026-08-29T00:15:24Z] RESULT temperature=0.7 run=8 latency_ms=25714.397 tags=["food safety", "product recall", "allergen warning"]
[2026-08-29T00:15:48Z] RESULT temperature=0.7 run=9 latency_ms=23997.9 tags=["food safety", "product recall", "allergy warning"]
[2026-08-29T00:16:14Z] RESULT temperature=0.7 run=10 latency_ms=25791.445 tags=["food recall", "allergen warning", "product safety"]
[2026-08-29T00:16:37Z] RESULT temperature=0.7 run=11 latency_ms=22960.303 tags=["food recall", "allergen warning", "product safety"]
[2026-08-29T00:17:02Z] RESULT temperature=0.7 run=12 latency_ms=25182.716 tags=["food recall", "allergen safety", "product return"]
[2026-08-29T00:17:26Z] RESULT temperature=0.7 run=13 latency_ms=23901.451 tags=["food safety", "allergen recall", "product recall"]
[2026-08-29T00:17:51Z] RESULT temperature=0.7 run=14 latency_ms=25571.28 tags=["food recall", "allergen warning", "product safety"]
[2026-08-29T00:18:17Z] RESULT temperature=0.7 run=15 latency_ms=25719.189 tags=["food recall", "allergy warning", "product safety"]
[2026-08-29T00:18:42Z] RESULT temperature=0.7 run=16 latency_ms=25535.034 tags=["food recall", "allergen warning", "product safety"]
[2026-08-29T00:19:05Z] RESULT temperature=0.7 run=17 latency_ms=22135.471 tags=["food recall", "allergen warning", "product safety"]
[2026-08-29T00:19:29Z] RESULT temperature=0.7 run=18 latency_ms=25826.043 tags=["food recall", "allergen safety", "product return"]
[2026-08-29T00:19:55Z] RESULT temperature=0.7 run=19 latency_ms=24654.838 tags=["food recall", "allergy warning", "product safety"]
[2026-08-29T00:20:19Z] RESULT temperature=0.7 run=20 latency_ms=23927.22 tags=["food recall", "allergy warning", "product safety"]
[2026-08-29T00:20:44Z] RESULT temperature=0.0 run=1 latency_ms=25128.169 tags=["food recall", "allergen warning", "product safety"]
[2026-08-29T00:21:07Z] RESULT temperature=0.0 run=2 latency_ms=23222.395 tags=["food recall", "allergen warning", "product safety"]
[2026-08-29T00:21:30Z] RESULT temperature=0.0 run=3 latency_ms=22853.474 tags=["food recall", "allergen warning", "product safety"]
[2026-08-29T00:21:53Z] RESULT temperature=0.0 run=4 latency_ms=22831.652 tags=["food recall", "allergen warning", "product safety"]
[2026-08-29T00:22:16Z] RESULT temperature=0.0 run=5 latency_ms=22921.963 tags=["food recall", "allergen warning", "product safety"]
[2026-08-29T00:22:39Z] RESULT temperature=0.0 run=6 latency_ms=23037.082 tags=["food recall", "allergen warning", "product safety"]
[2026-08-29T00:23:02Z] RESULT temperature=0.0 run=7 latency_ms=22770.136 tags=["food recall", "allergen warning", "product safety"]
[2026-08-29T00:23:25Z] RESULT temperature=0.0 run=8 latency_ms=23068.115 tags=["food recall", "allergen warning", "product safety"]
[2026-08-29T00:23:48Z] RESULT temperature=0.0 run=9 latency_ms=22800.544 tags=["food recall", "allergen warning", "product safety"]
[2026-08-29T00:24:11Z] RESULT temperature=0.0 run=10 latency_ms=22960.627 tags=["food recall", "allergen warning", "product safety"]
[2026-08-29T00:24:34Z] RESULT temperature=0.0 run=11 latency_ms=23258.206 tags=["food recall", "allergen warning", "product safety"]
[2026-08-29T00:24:57Z] RESULT temperature=0.0 run=12 latency_ms=23194.989 tags=["food recall", "allergen warning", "product safety"]
[2026-08-29T00:25:20Z] RESULT temperature=0.0 run=13 latency_ms=22927.966 tags=["food recall", "allergen warning", "product safety"]
[2026-08-29T00:25:43Z] RESULT temperature=0.0 run=14 latency_ms=22711.064 tags=["food recall", "allergen warning", "product safety"]
[2026-08-29T00:26:05Z] RESULT temperature=0.0 run=15 latency_ms=22725.813 tags=["food recall", "allergen warning", "product safety"]
[2026-08-29T00:26:28Z] RESULT temperature=0.0 run=16 latency_ms=22831.228 tags=["food recall", "allergen warning", "product safety"]
[2026-08-29T00:26:51Z] RESULT temperature=0.0 run=17 latency_ms=22725.813 tags=["food recall", "allergen warning", "product safety"]
[2026-08-29T00:27:13Z] RESULT temperature=0.0 run=18 latency_ms=22511.162 tags=["food recall", "allergen warning", "product safety"]
[2026-08-29T00:27:36Z] RESULT temperature=0.0 run=19 latency_ms=22806.988 tags=["food recall", "allergen warning", "product safety"]
[2026-08-29T00:27:59Z] RESULT temperature=0.0 run=20 latency_ms=23056.728 tags=["food recall", "allergen warning", "product safety"]
[2026-08-29T00:27:59Z] COMPLETE results=reports\hw01\raw\nondeterminism_results.json
{
  "0.7": {
    "runs": 20,
    "distinct_tag_sets": 6,
    "tags_in_all_runs": [],
    "tags_in_exactly_one_run": [
      "allergen recall"
    ],
    "latency_ms": {
      "p50": 25716.793,
      "p95": 35811.859,
      "p99": 62209.399
    }
  },
  "0.0": {
    "runs": 20,
    "distinct_tag_sets": 1,
    "tags_in_all_runs": [
      "allergen warning",
      "food recall",
      "product safety"
    ],
    "tags_in_exactly_one_run": [],
    "latency_ms": {
      "p50": 22807.268,
      "p95": 23351.704,
      "p99": 24772.876
    }
  }
}
PS C:\Users\shara\data260-9619>
```

Part 4 | Model Client and Token Accounting

I created a reusable model adapter in `src/model_client.py` with the stable interface `complete(messages, tools=None)`. All Ollama requests pass through this adapter, so the connection and token-counting logic are kept in one place. I created `code/hw1_client.py` as a command-line demonstration. It imports the adapter, sends five code-review requests to `qwen3:8b`, and records the input, output, and total tokens for every response.

The instructions in `AGENT.md` require the model to return strict bullet-only code reviews. All five responses followed this format successfully.

Command Used

```
python scripts/recorded_run.py -- python code/hw1_client.py `
--demo `
--model qwen3:8b `
--output-json reports/hw01/raw/client_token_counts.json
```

Per-Turn Token Counts

Turn	Input Tokens	Output Tokens	Total Tokens
1	198	23	221
2	244	24	268
3	290	30	320
4	342	29	371
5	395	26	421

/stats Results

Statistic	After Turn 3	After Turn 5
Turn count	3	5
Cumulative input tokens	732	1,469
Cumulative output tokens	77	132
Cumulative total tokens	809	1,601
Serialized history length	1,554	2,069

The `/stats` command displays the current totals without adding anything to the conversation history.

Required Questions

1. Why is previous conversation context resent with every turn?

Model requests are stateless, meaning the model does not automatically remember earlier requests. The previous messages must be sent again so the model can understand the conversation.

2. How is a system prompt different from a user message?

A system prompt gives the model its main behavior and rules. A user message contains the current question or task.

3. Why do input tokens grow during a conversation?

Each new request includes more previous messages. As the conversation history becomes longer, the number of input tokens increases.

4. What eventually limits that growth?

Growth is limited by the model's context window, which is the maximum amount of text the model can process at one time. Applications may also shorten or summarize older messages.

The screenshots below show the token statistics after turns 3 and 5.

```
PS C:\Users\shara\data260-9619> Write-Host " Turn 3 token counts "
Turn 3 token counts
PS C:\Users\shara\data260-9619> $client.turns[2] | Select-Object turn,input_tokens,output_tokens,total_tokens,bullet_only | Format-List

turn          : 3
input_tokens   : 290
output_tokens  : 30
total_tokens   : 320
bullet_only    : True

PS C:\Users\shara\data260-9619> Write-Host " /stats after turn 3 "
/stats after turn 3
PS C:\Users\shara\data260-9619> $client.stats_snapshots.PSObject.Properties["3"].Value | Format-List

turn_count      : 3
input_tokens     : 732
output_tokens    : 77
total_tokens     : 809
serialized_history_length : 1554

PS C:\Users\shara\data260-9619>
```

```

PS C:\Users\shara\data260-9619> Write-Host "Total runs:" $experiment.runs.Count
Total runs: 40
PS C:\Users\shara\data260-9619> $experiment.runs | Group-Object temperature | Select-Object Name,Count

Name Count
----
0.7 20
0.8 20

PS C:\Users\shara\data260-9619> Write-Host " Measured metrics "
Measured metrics
PS C:\Users\shara\data260-9619> Get-Content .\reports\hw01\METRICS.md
# Homework 1 Experiment Metrics

- Model: 'gwen3:8b'
- Generated (UTC): '2026-08-29T00:27:59Z'
- Runs per temperature: 20
- Percentile method: linear interpolation between adjacent ordered observations

## Tag variation

| Metric | Temp 0.7 | Temp 0.8 |
| --- | --- | --- |
| Distinct tag sets | 6 | 1 |
| Tags in all runs | None | 'allergen warning', 'food recall', 'product safety' |
| Tags in exactly one run | 'allergen recall' | None |

## Latency

| Metric | Temp 0.7 | Temp 0.8 |
| --- | --- | --- |
| p50 (ms) | 25716.793 | 22887.268 |
| p95 (ms) | 35811.859 | 23351.704 |
| p99 (ms) | 62209.399 | 24772.876 |

## Interpretation

Replace this paragraph with your interpretation of the measured values. Explain what two users
sending the fixed input might see, then give one acceptable and one unacceptable example of
run-to-run variation.

```

```

PS C:\Users\shara\data260-9619> Write-Host " Turn 5 token counts "
Turn 5 token counts
PS C:\Users\shara\data260-9619> $client.turns[4] | Select-Object turn,input_tokens,output_tokens,total_tokens,bullet_only | Format-List

turn          : 5
input_tokens   : 395
output_tokens  : 26
total_tokens   : 421
bullet_only    : True

PS C:\Users\shara\data260-9619> Write-Host " /stats after turn 5 "
/stats after turn 5
PS C:\Users\shara\data260-9619> $client.stats_snapshots.PSObject.Properties["5"].Value | Format-List

turn_count          : 5
input_tokens        : 1469
output_tokens       : 132
total_tokens        : 1601
serialized_history_length : 2869

PS C:\Users\shara\data260-9619> Write-Host " Cumulative tokens on exit "
Cumulative tokens on exit
PS C:\Users\shara\data260-9619> $client.cumulative | Format-List

turn_count          : 5
input_tokens        : 1469
output_tokens       : 132
total_tokens        : 1601

PS C:\Users\shara\data260-9619>

```

Additional Screenshots during execution:

Docker

```
PS C:\Users\shara\data260-9619> docker version
Client:
Version:           29.3.1
API version:       1.54
Go version:        go1.25.8
Git commit:        c2be9cc
Built:             Wed Mar 25 16:16:33 2026
OS/Arch:           windows/amd64
Context:           desktop-linux

Server: Docker Desktop 4.67.0 (222858)
Engine:
Version:           29.3.1
API version:       1.54 (minimum version 1.40)
Go version:        go1.25.8
Git commit:        f78c987
Built:             Wed Mar 25 16:13:48 2026
OS/Arch:           linux/amd64
Experimental:      false
Containerd:
Version:           v2.2.1
GitCommit:         dea7da592f5d1d2b7755e3a161be07f43fad8f75
runc:
Version:           1.3.4
GitCommit:         v1.3.4-0-gd6d73eb8
docker-init:
Version:           0.19.0
GitCommit:         de40ad0

PS C:\Users\shara\data260-9619> docker compose up --build -d
#1 [internal] load local bake definitions
#1 [internal] load local bake definitions
#1 reading from stdin 519B 0.0s done
#1 DONE 0.0s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile:
#2 transferring dockerfile: 342B 0.0s done
#2 DONE 0.2s

#3 [internal] load metadata for docker.io/library/nginx:1.27-alpine
#3 ...
```

```
#7 DONE 3.2s

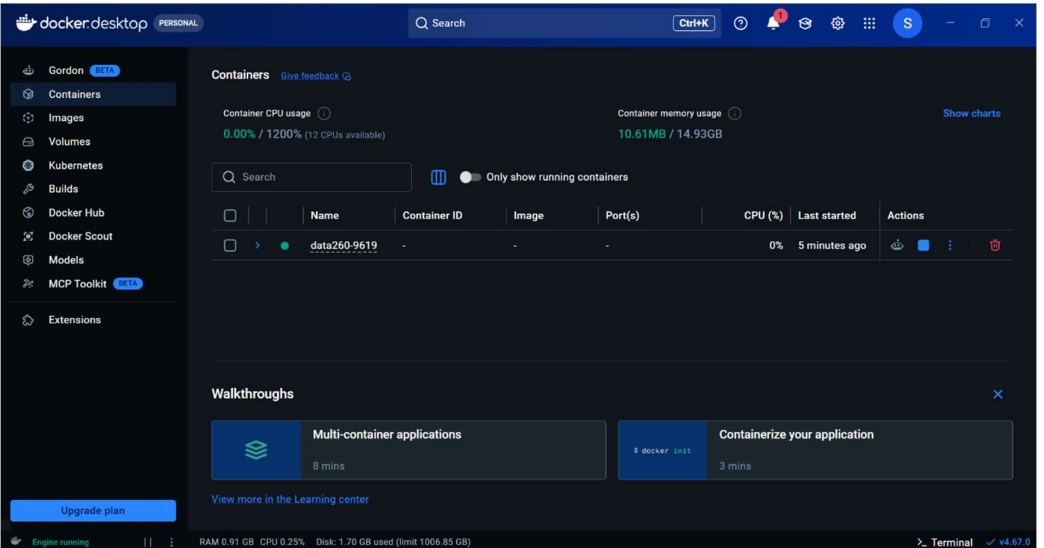
#7 [1/3] FROM docker.io/library/nginx:1.27-alpine@sha256:65645c7bb6a0661892a8b03b89d0743208a18dd2f3f17a54ef4b76fb8e2f2a10
#7 extracting sha256:197eb75867ef4fced4724f17b0972ab0489436860a594a9445f8eaff8155053 0.0s done
#7 extracting sha256:34a6464b756511a2e217f0508e11d1a572085d66cd6cd9a555a802ad49a3102 0.0s done
#7 extracting sha256:39c2ddfd6018082a4a646e7ca44e95aca9bf3aebc00f17f7ccc2954804f2a7d
#7 extracting sha256:39c2ddfd6018082a4a646e7ca44e95aca9bf3aebc00f17f7ccc2954804f2a7d 2.4s done
#7 DONE 5.7s

#8 [2/3] RUN rm -rf /usr/share/nginx/html/*
#8 DONE 1.2s

#9 [3/3] COPY index.html styles.css app.js /usr/share/nginx/html/
#9 DONE 0.1s

#10 exporting to image
#10 exporting layers
#10 exporting layers 0.2s done
#10 exporting manifest sha256:983a8cf8bac41677d70e3856f0b6ab15dff69ea788a258b87b2f3858b61a28e6 0.0s done
#10 exporting config sha256:2a538df9dce5b-e74bfcbaace30b9bf7223bf9d0db6caf39709786da2d0ddc 0.0s done
#10 exporting attestation manifest sha256:b9bffa43a8116ee50986d813a78cda934526b96763a0119007da6922dc04e08 0.1s done
#10 exporting manifest list sha256:2e605a3b06203d6818d863c620b8811e5361598f7fe60481592af3a2a476efae
#10 exporting manifest list sha256:2e605a3b06203d6818d863c620b8811e5361598f7fe60481592af3a2a476efae 0.0s done
#10 naming to docker.io/library/data260-hw1-s9619:latest done
#10 unpacking to docker.io/library/data260-hw1-s9619:latest 0.1s done
#10 DONE 0.6s

#11 resolving provenance for metadata file
#11 DONE 0.0s
[+] up 9/3
✓ Image data260-hw1-s9619:latest Built 11.6s
✓ Network data260-9619_default Created 0.1s
✓ Container data260-hw1-s9619 Started 1.4s
PS C:\Users\shara\data260-9619> docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
data260-hw1-s9619   data260-hw1-s9619:latest   "/docker-entrypoint..." web        17 seconds ago Up 16 seconds (healthy)   0.0.0.0:8619->80/tcp, [::]:8619->80/tcp
PS C:\Users\shara\data260-9619> (Invoke-WebRequest -Uri http://localhost:8619 -UseBasicParsing).StatusCode
200
PS C:\Users\shara\data260-9619> Start-Process "http://localhost:8619"
PS C:\Users\shara\data260-9619>
```




```

PS C:\Users\shara\data260-9619> aws ecr get-login-password --region us-east-1 |
>> docker login --username AWS --password-stdin $Registry
Login Succeeded
PS C:\Users\shara\data260-9619> docker build --platform linux/amd64 -f code/Dockerfile -t my-web-app:latest code
[+] Building 2.1s (9/9) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 398B
=> [internal] load metadata for docker.io/library/nginx:1.27-alpine
=> [auth] library/nginx:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 99B
=> [1/3] FROM docker.io/library/nginx:1.27-alpine@sha256:65645c7bb6a0661892a8b03b89d0743208a18dd2f3f17a54ef4b76fb8e2f2a10
=> => resolve docker.io/library/nginx:1.27-alpine@sha256:65645c7bb6a0661892a8b03b89d0743208a18dd2f3f17a54ef4b76fb8e2f2a10
=> [internal] load build context
=> => transferring context: 171B
=> CACHED [2/3] RUN rm -rf /usr/share/nginx/html/*
=> CACHED [3/3] COPY web_application/index.html web_application/styles.css web_application/app.js /usr/share/nginx/html/
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:9162a35c1e303c070dbeefef37a557b9b7799bdfdf743025939501cc023b339af
=> => exporting config sha256:143db92d090f636b063565dc4bbeb47fabad10e713fca350e739aedaa98dd469
=> => exporting attestation manifest sha256:076762593610e2d1e33482a11ff03a4befe914d23096fc6f6614162c8e54f7dd
=> => exporting manifest list sha256:4849af932180e02c2f0cc915e5e77dd40c1c21cd8c650564c8d73eb822c5238b
=> => naming to docker.io/library/my-web-app:latest
=> => unpacking to docker.io/library/my-web-app:latest
PS C:\Users\shara\data260-9619> docker tag my-web-app:latest "${EcrUri}:latest"
PS C:\Users\shara\data260-9619> docker push "${EcrUri}:latest"
The push refers to repository [572630832153.dkr.ecr.us-east-1.amazonaws.com/my-web-app]
39c2ddfd6018: Pushed
bb8cb5f52d1b: Pushed
d7e507024086: Pushed
b464cfd7a63: Pushed
34a64644b756: Pushed
61b8ed7ec67: Pushed
197eb75867ef: Pushed
5520bda330dc: Pushed
5ef14a6d4c3d: Pushed
f18232174bc9: Pushed
61ca4f733c80: Pushed
latest: digest: sha256:4849af932180e02c2f0cc915e5e77dd40c1c21cd8c650564c8d73eb822c5238b size: 856
PS C:\Users\shara\data260-9619>

```